



NATIONAL UNIVERSITY OF SINGAPORE

COLLEGE OF DESIGN AND ENGINEERING

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

EE4308 Autonomous Robot Systems

Project 2: Drone Simulation with Simplified Kalman Filter

TEAM 06

A0319170J | A0318793N | A0326802J

April 7, 2026

Contents

1	Behavior	3
1.1	Algorithm Examination	3
1.1.1	Finite State Machine Structure	3
1.1.2	Waypoint Reaching	3
1.2	Improvements	3
1.2.1	Live Waypoint Tracking for TURTLE_POSITION	3
1.2.2	Cycle-Completion Guarantee	4
1.2.3	Path Availability Guard	4
1.3	Parameter Tuning	4
2	Controller	5
2.1	Controller Logic	5
2.2	Analysis of the Existing Algorithm	6
2.3	Justification of Parameter Choices	8
2.4	Possible Improvements	8
2.5	Discussion	9
2.6	Summary	9
3	Estimator	9
3.1	Algorithm Examination	9
3.1.1	State Decomposition	10
3.1.2	Prediction Step	10
3.1.3	Correction Models	11
3.2	Improvements	11
3.2.1	Sonar Gating	11
3.2.2	Barometer Bias State	11
3.2.3	GPS Velocity Correction	12
3.2.4	GPS Forward Compensation	12
3.3	Parameter Tuning	12
3.4	Representative Comparison	12
3.5	Final Performance	14
4	Contribution	14
5	AI Tool Declaration	18

1 Behavior

1.1 Algorithm Examination

1.1.1 Finite State Machine Structure

The behavior node coordinates the drone’s mission using a seven-state FSM. Table 1 summarises the states, their waypoints, and the conditions under which they transition.

Table 1: FSM states, waypoints, and transition conditions.

State	Waypoint	Transition condition	Next state
BEGIN	—	On startup	TAKEOFF
TAKEOFF	(x_0, y_0, h_c)	Reached & turtle stopped	INITIAL
		Reached & turtle running	TURTLE_POSITION
TURTLE_POSITION	turtle position at h_c	Reached	TURTLE_WAYPOINT
TURTLE_WAYPOINT	turtle goal at h_c	Reached	INITIAL
INITIAL	(x_0, y_0, h_c)	Reached & turtle stopped	LANDING
		Reached & turtle running	TURTLE_POSITION
LANDING	(x_0, y_0, z_0)	Reached	END
END	—	Motors off, timer cancelled	—

The drone repeats the three-step cycle $TURTLE_POSITION \rightarrow TURTLE_WAYPOINT \rightarrow INITIAL$ until the turtlebot stops. The `turtle_stop_` flag is checked only at INITIAL (and TAKEOFF), so the cycle always finishes before the landing decision is made.

1.1.2 Waypoint Reaching

A waypoint is reached when the 3D Euclidean distance drops below the threshold r_{thres} :

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} < r_{\text{thres}}$$

Using the full 3D distance rather than 2D prevents the drone from triggering arrival while it is still climbing or descending toward the target altitude.

1.2 Improvements

1.2.1 Live Waypoint Tracking for TURTLE_POSITION

The template sets the waypoint once at the moment of state entry. The turtlebot moves at up to 0.22 m/s; at a behavior timer frequency of 10 Hz, it travels roughly 0.022 m per tick. Over the several seconds the drone takes to fly to the turtle, the turtlebot can

displace by 0.5–1 m. Chasing a stale waypoint causes the drone to land in empty space and immediately transition to `TURTLE_WAYPOINT`, skipping the actual interception. The fix updates the waypoint on every timer tick while in `TURTLE_POSITION`:

```
if (state_ == TURTLE_POSITION && !turtle_plan_.poses.empty())
{
    waypoint_x_ = turtle_plan_.poses.front().pose.position.x;
    waypoint_y_ = turtle_plan_.poses.front().pose.position.y;
    waypoint_z_ = cruise_height_;
}
```

The plan request that follows in the same tick then generates a fresh path to the updated target, keeping the controller on course.

1.2.2 Cycle-Completion Guarantee

If `turtle_stop_` were checked at every state, the drone could abort mid-cycle (e.g. during `TURTLE_WAYPOINT`) and attempt to land from an arbitrary position. Instead, the flag is evaluated only at `INITIAL` and `TAKEOFF`. This guarantees the drone completes the current visit before landing, matching the mission requirement that the cycle must complete.

An additional edge case arises if the turtlebot stops before the drone finishes taking off. In this case the drone transitions `TAKEOFF` $\rightarrow$ `INITIAL` $\rightarrow$ `LANDING`, skipping the cycle entirely rather than performing a vacuous visit to a stationary turtle.

1.2.3 Path Availability Guard

`TURTLE_PLAN_` is populated only once the turtlebot begins moving. Both `TURTLE_POSITION` and `TURTLE_WAYPOINT` transitions guard against an empty path before reading `poses.front()` or `poses.back()`, which would otherwise cause undefined behavior at startup.

1.3 Parameter Tuning

Table 2: Behavior node parameters.

Parameter	Value	Unit	Constraint
<code>cruise_height</code>	5.0	m	Must be 5.0
<code>reached_thres</code>	0.3	m	≤ 0.3
<code>frequency</code>	10.0	Hz	—

reached_thres. Set to 0.3 m, the maximum permitted value, to minimise hover time at each waypoint. The same threshold applies to all states including LANDING, so the drone may touch down up to 0.3 m from the target ground position. This is acceptable in simulation; a mission requiring precise landing would need a separate, tighter tolerance for the LANDING state.

frequency. Set to 10 Hz. At this rate, the waypoint update lag is at most 0.1 s, giving a maximum live-tracking error of 0.022 m per tick from turtle motion. This is well within the 0.3 m arrival threshold, so increasing the frequency is not necessary.

2 Controller

The controller converts the planned path into velocity commands for the drone. In this project, the drone is holonomic, so it can move directly in the horizontal and vertical directions. Because of this, the curvature-based pure pursuit method used for non-holonomic robots is not necessary here. Instead, the controller follows a simpler lookahead-based strategy, where the drone moves toward a selected point on the path while rotating at a constant yaw rate.

2.1 Controller Logic

The controller runs at a fixed frequency and generates one velocity command at each timer callback. It first checks whether the controller is enabled. If not, the function returns immediately. It then checks whether odometry has been received and whether the planned path is non-empty. If either of these is unavailable, the controller publishes zero velocity so that the drone remains stationary.

When both odometry and path data are available, the controller finds the path point that is closest to the current drone position. Starting from that point, it searches forward along the path until it finds the first point whose distance from the drone is greater than or equal to the lookahead distance. This point is used as the lookahead point. If no such point is found, the last point on the path is used instead.

Let the current drone position in the world frame be (x, y, z) and the selected lookahead point be (x_L, y_L, z_L) . The translational velocity command in the world frame is computed using proportional control:

$$v_x^w = k_{p,xy}(x_L - x), \quad v_y^w = k_{p,xy}(y_L - y), \quad v_z = k_{p,z}(z_L - z). \quad (1)$$

The horizontal velocity is then limited such that

$$\sqrt{(v_x^w)^2 + (v_y^w)^2} \leq v_{xy,\max}, \quad (2)$$

and the vertical velocity is clamped such that

$$|v_z| \leq v_{z,\max}. \quad (3)$$

Since the command must be published in the drone body frame, the horizontal velocity is rotated from the world frame into the body frame using the current yaw angle ψ :

$$\begin{bmatrix} v_x^b \\ v_y^b \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} v_x^w \\ v_y^w \end{bmatrix}. \quad (4)$$

Finally, the controller publishes the translational velocity together with a constant yaw rate:

$$\omega_z = \text{yaw_vel}. \quad (5)$$

This is consistent with the project requirement that the drone should keep rotating clockwise while moving. The overall control procedure is shown in Fig. 1. The flowchart summarizes the main processing stages of the controller, namely state checking, lookahead point selection, velocity generation, saturation, frame transformation, and command publication.

2.2 Analysis of the Existing Algorithm

The controller is designed to be simple and easy to implement. This is reasonable for the current project because the main requirement is reliable path following in simulation rather than high-performance flight control.

One advantage of the current design is that it is straightforward. The controller only depends on odometry and the planned path, and the logic is relatively easy to debug. This is useful in a course project, since it reduces implementation complexity and makes it easier to identify problems in either the planner, controller, or estimator.

Another advantage is that the use of a lookahead point gives smoother motion than directly commanding the drone toward the final goal at every time step. Instead of reacting only to the endpoint, the drone is guided along the path progressively. This makes the motion more natural and generally improves path tracking.

The explicit velocity limits are also important. By constraining horizontal and vertical motion separately, the controller satisfies the project requirements and avoids overly aggressive commands. This is especially useful for altitude control, where excessive vertical motion tends to cause more visible oscillation.

However, the controller also has some limitations. First, the lookahead distance is fixed. This means the same value is used for straight motion, turning, and final approach. In practice, a larger lookahead usually gives smoother motion but tends to cut corners, while a smaller lookahead improves path accuracy but can lead to sharper corrections. Second, the controller uses only proportional feedback based on position error. It does not explicitly account for the drone's current velocity, and it does not predict future

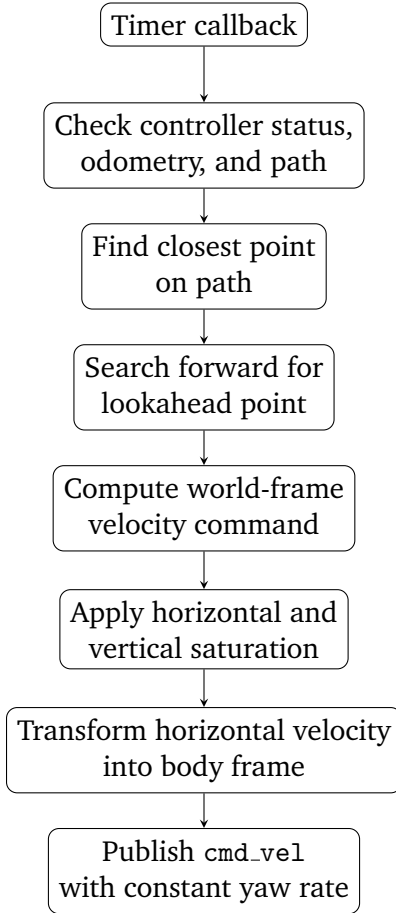


Figure 1: Overall flow of the controller. At each timer callback, the controller checks whether valid odometry and path data are available, selects a lookahead point, computes the translational velocity command, applies velocity limits, transforms the command into the body frame, and publishes the final control input.

motion. As a result, the response may not be as smooth as a more advanced controller, particularly when the drone is close to the target.

Third, the closest path point is chosen based only on Euclidean distance. This works well enough for short and simple paths, but it may become less reliable if the drone deviates significantly from the path or if the path shape becomes more complicated.

Finally, the yaw motion is fully separated from translational path following. This is acceptable here because the yaw rate is specified as a fixed requirement, but it means the controller does not actively choose an orientation that helps tracking performance.

Overall, the current controller is a reasonable baseline solution. It is simple, functional, and suitable for the scope of the project, although it is not intended to be a highly refined tracking controller.

2.3 Justification of Parameter Choices

The main parameters in the controller are the lookahead distance, maximum horizontal velocity, maximum vertical velocity, yaw velocity, and the proportional gains for horizontal and vertical motion.

The lookahead distance determines how far ahead on the path the controller tries to follow. If the value is too small, the drone may react too strongly to small path changes or estimation noise. If the value is too large, the drone may cut corners and become less accurate when approaching waypoints. For this reason, a moderate lookahead distance is more suitable.

The maximum horizontal and vertical velocities are chosen according to the project constraints. These limits are important because they directly affect the safety and smoothness of the motion. In particular, the vertical velocity limit should remain relatively small, since altitude changes are usually more sensitive and more likely to oscillate.

The proportional gain in the horizontal plane should be high enough for the drone to respond promptly to the path, but not so high that it produces unstable or oscillatory motion. Similarly, the vertical gain should be chosen more conservatively, since vertical motion is more likely to overshoot when the gain is too large.

The yaw velocity must also be chosen carefully, since the project specifies a constant clockwise rotation rate. Because ROS uses radians per second, the desired angular speed must be converted correctly from degrees per second.

2.4 Possible Improvements

Although the current controller works reasonably well, several simple improvements could make it perform better without changing the overall structure too much.

One possible improvement is to use an adaptive lookahead distance. For example, the controller could use a larger lookahead when the drone is far from the goal and a smaller lookahead when it is close to a waypoint. This would help balance smooth motion and tracking accuracy.

Another possible improvement is to reduce the commanded speed more deliberately when the drone is near the final target. While proportional control already decreases the velocity as the position error becomes smaller, an additional speed scheduling rule could make the final approach smoother.

A third improvement is to keep track of the previously selected closest path index and continue searching from there, instead of searching the full path every time. This would reduce the chance of the controller jumping backward to an earlier point on the path.

A small deadband around the target position or target altitude could also help. Once the drone is sufficiently close to the desired point, very small corrections may not be necessary and may only introduce visible jitter due to noise.

These improvements are relatively simple and can be added without replacing the basic controller design.

2.5 Discussion

From a practical point of view, the current controller achieves a good balance between simplicity and functionality. It satisfies the main project requirements, namely path following, velocity limiting, and constant yaw rotation. It is also easy to understand and implement, which is important in a course project where the emphasis is not only on performance but also on correctness and clarity.

At the same time, the controller still behaves like a basic proportional path follower. It does not adapt itself to different path geometries, and it does not explicitly optimize tracking accuracy or smoothness. Therefore, while it is sufficient for the current simulation task, it should be seen as a simple baseline rather than a complete flight control solution.

2.6 Summary

In summary, the controller uses a lookahead-based path-following method that suits a holonomic drone well. The implementation is straightforward, respects the required motion constraints, and is sufficient for the project task. Its main strengths are simplicity and ease of tuning, while its main weaknesses are the use of a fixed lookahead distance and a basic proportional control law. Even so, the controller provides a practical and effective solution for this project.

3 Estimator

3.1 Algorithm Examination

The handout already gives the basic filter structure, so here we only keep the parts needed to explain our final implementation and our changes.

3.1.1 State Decomposition

We keep the axis-by-axis structure from the handout. Instead of one large coupled filter, we use four small filters:

$$X_x = [x, v_x]^T, \quad X_y = [y, v_y]^T, \quad X_z = [z, v_z, b_{\text{baro}}]^T, \quad X_a = [\psi, \dot{\psi}]^T$$

where b_{baro} is the barometer bias and ψ is yaw. This is still a simple estimator, not a full 3D EKF.

3.1.2 Prediction Step

The IMU callback performs the prediction step. Let (u_x, u_y, u_z) be the linear acceleration from the IMU in the drone frame. We rotate the horizontal acceleration into the world frame using the current yaw:

$$a_x = \cos \psi u_x - \sin \psi u_y, \quad a_y = \sin \psi u_x + \cos \psi u_y$$

Then the x and y filters use a constant-acceleration model:

$$\begin{aligned} x_{k+1} &= x_k + v_{x,k} \Delta t + \frac{1}{2} a_x \Delta t^2 \\ v_{x,k+1} &= v_{x,k} + a_x \Delta t \end{aligned}$$

The same form is used for y .

For the vertical axis, we first remove gravity:

$$a_z = u_z - g$$

In the code, a_z is also clamped to $\pm 1.5 \text{ m/s}^2$ to limit IMU spikes. The z state is predicted by

$$\begin{aligned} z_{k+1} &= z_k + v_{z,k} \Delta t + \frac{1}{2} a_z \Delta t^2 \\ v_{z,k+1} &= v_{z,k} + a_z \Delta t \\ b_{\text{baro},k+1} &= b_{\text{baro},k} \end{aligned}$$

Yaw is predicted from the IMU angular velocity:

$$\psi_{k+1} = \psi_k + \omega_z \Delta t$$

and the angle is wrapped after each update.

Table 3: Measurement models used by the estimator.

Sensor	Measurement	Model	Updated state
GPS	position	$z_{\text{gps}} = [x, y, z]^T$	X_x, X_y, X_z
Sonar	height	$z_{\text{sonar}} = z$	X_z
Barometer	altitude	$z_{\text{baro}} = z + b_{\text{baro}}$	X_z
Magnetometer	yaw	$z_{\text{mag}} = \psi$	X_a

3.1.3 Correction Models

The correction step is also separated by sensor. Table 3 shows the measurement models used in the final code.

The first GPS message initializes the local origin. After that, GPS measurements are converted from latitude, longitude, and altitude into ECEF, then into a local NED frame, and finally into Gazebo’s ENU map frame. The magnetometer yaw is computed by

$$\psi_{\text{mag}} = \text{atan2}(-m_y, m_x)$$

The barometer altitude is computed from pressure by

$$z_{\text{baro}} = 44330 \left(1 - \left(\frac{P}{101325} \right)^{0.1903} \right)$$

For scalar corrections, we update the covariance with the Joseph form:

$$P^+ = (I - KH)P^-(I - KH)^T + KRK^T$$

We keep this form for numerical stability.

3.2 Improvements

3.2.1 Sonar Gating

In our early runs, the sonar helped near the ground but became unreliable higher up. In the final code, we only accept the sonar update when the predicted height is at most 3.8 m and the innovation is at most 1.0 m.

3.2.2 Barometer Bias State

We observed a noticeable barometer offset. Treating it as a direct measurement of z gave a biased height estimate. In the final version, we add a bias term b_{baro} into the vertical state and use

$$H_{\text{baro}} = [1 \ 0 \ 1]$$

The first barometer message initializes the bias by $b_{\text{baro}} = z_{\text{baro}} - z$. After that, the filter can absorb the slow offset into the bias state instead of pushing it into the altitude estimate.

3.2.3 GPS Velocity Correction

Our main problem in the horizontal plane was lag. The estimate was usually stable, but it tended to stay behind the true motion. To reduce this, the estimator computes a planar pseudo-velocity from two consecutive GPS positions:

$$\hat{v}_{\text{gps}} = \frac{p_k - p_{k-1}}{\Delta t}$$

This velocity is smoothed and then used as a correction for v_x and v_y . We keep this update conservative. It is only used when the GPS time gap is reasonable and when the velocity innovation is not too large.

3.2.4 GPS Forward Compensation

GPS messages may arrive slightly later than the latest predicted state. If we correct with an old GPS position directly, the estimate stays behind the true motion. To reduce this mismatch, we forward-compensate the GPS position using the current estimated velocity:

$$p_{\text{gps,corr}} = p_{\text{gps}} + \hat{v} \Delta t_{\text{lag}}$$

This correction is only applied when the lag is positive and smaller than 0.5 s.

3.3 Parameter Tuning

Table 4 lists the main estimator parameters used in the final submission. We only include the parameters that directly affect the final behavior. The final values were chosen to improve the observed height error and horizontal lag, not to copy one lucky run.

Our tuning logic is simple. The z axis is mainly stabilized by sonar gating and the barometer bias state, so we keep the vertical IMU variance large. The largest remaining error is still in x and y , so we keep the GPS position variance smaller and leave the pseudo-velocity correction enabled.

3.4 Representative Comparison

Table 5 shows a before-and-after comparison for the z axis. The baseline comes from an early 40-second sweep, and the improved result comes after the z -axis fixes. We use this table to show that the main height failure was removed.

Table 4: Main estimator parameters.

Parameter	Value	Reason
frequency	30.0 Hz	Faster odom updates
var_imu_x, var_imu_y	1.5	Stable horizontal tuning
var_imu_z	20.0	Reduce trust in vertical IMU prediction
var_gps_x, var_gps_y	0.15	Reduce x/y lag
var_gps_z	0.5	GPS height is secondary
var_sonar	0.03	Strong low-altitude correction
var_baro	1.0	Moderate barometer trust
gps_forward_compensation_max_dt	0.5 s	Limit stale GPS compensation
gps_velocity_alpha	0.9	Smooth GPS pseudo-velocity
gps_velocity_variance_scale	0.1	Keep velocity correction light
gps_velocity_min_variance	0.04	Avoid over-trusting pseudo-velocity
gps_velocity_max_innovation	1.5	Reject inconsistent velocity updates

Table 5: Representative before-and-after comparison for the z axis.

Case	All-data z MAE	7–25 s z MAE
Early baseline	0.649 m	0.533 m
After z -axis fixes	0.020 m	0.019 m
Repeat run after fixes	—	0.020 m

This comparison supports our main claim for the vertical axis. The main change is in the sensor model: unreliable high-altitude sonar updates are rejected, and barometer bias is estimated as a state.

Figure 2 shows the same comparison visually. In the early baseline, the estimated height drops far below the true value several times after about 16 s. After the z -axis fixes, the height trace stays close to ground truth.

3.5 Final Performance

Table 6 shows the result of the final validation experiment. The main conclusion is that the z and yaw estimates are already quite accurate, while x and y are still the main limitation.

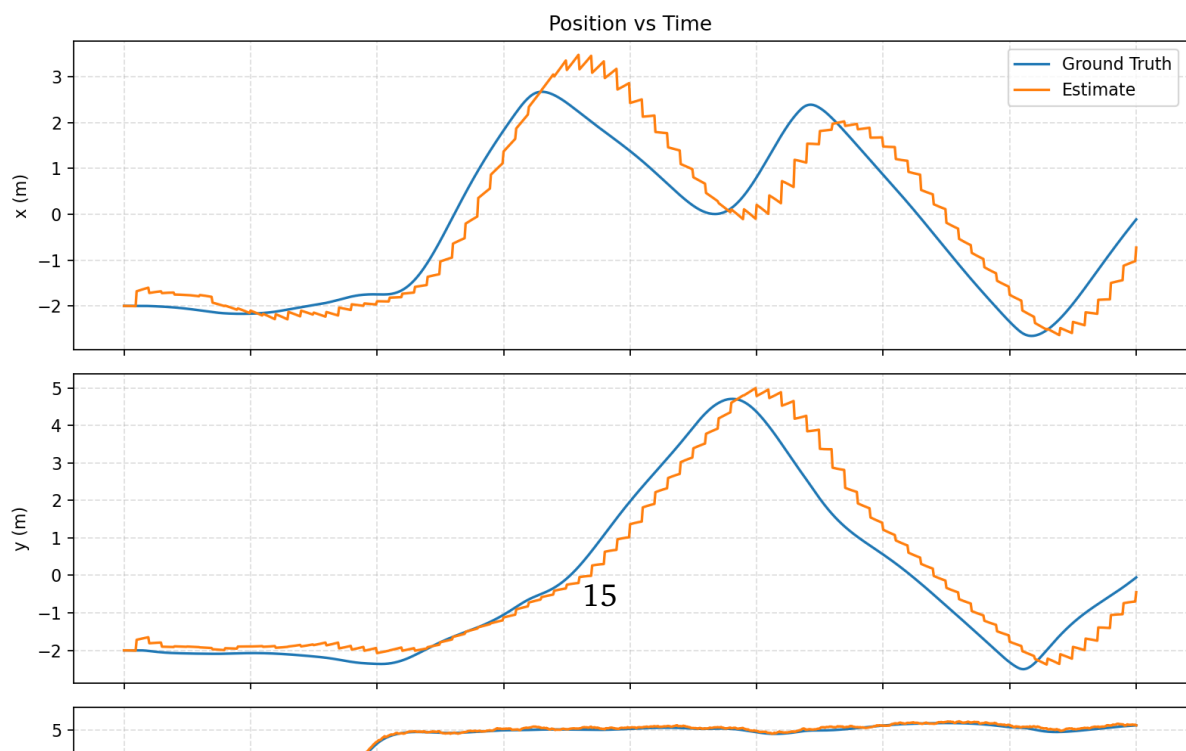
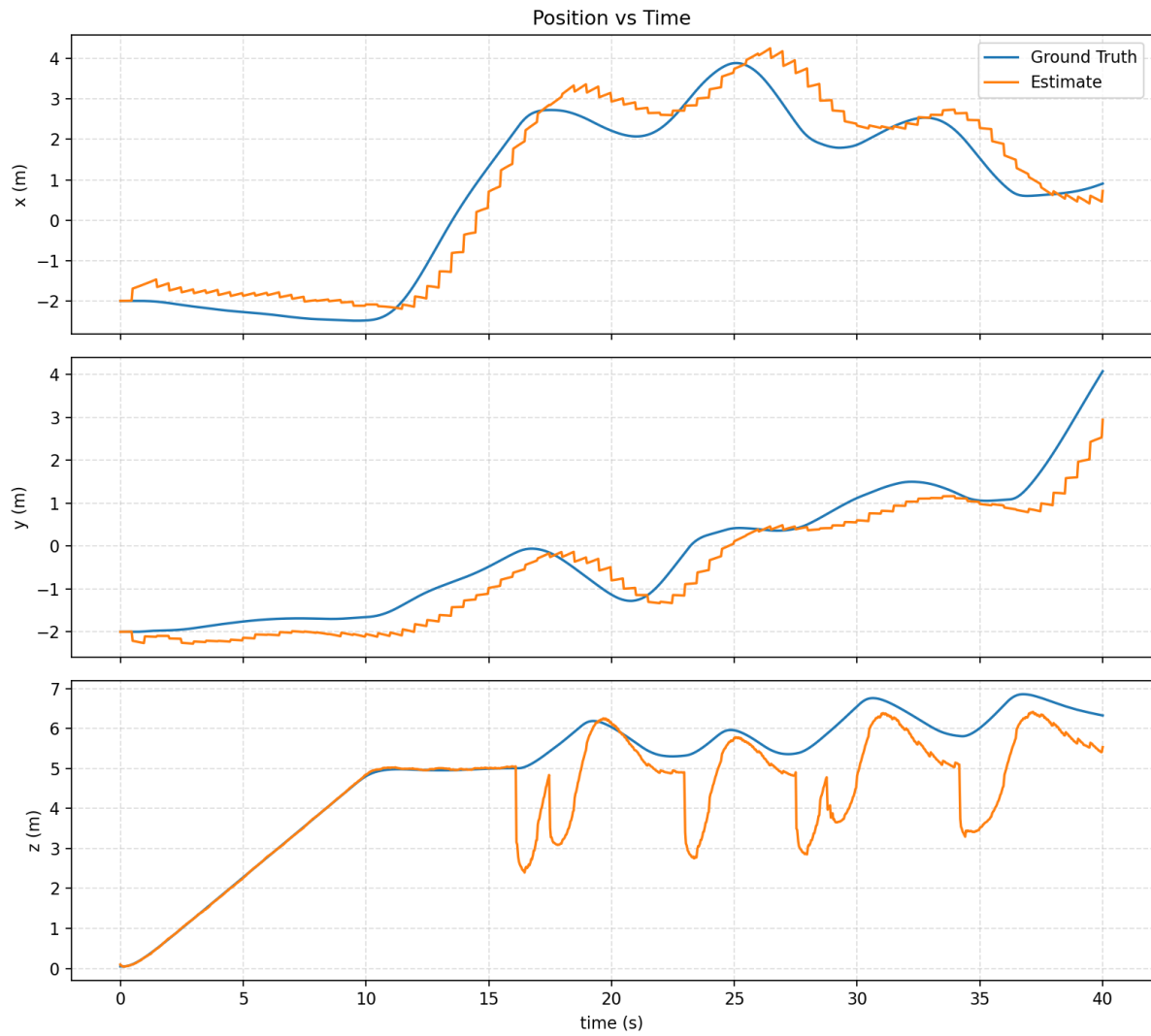
Table 6: Final validation result.

Metric	x	y	z	yaw
MAE	0.2099 m	0.3018 m	0.0199 m	0.00435 rad
RMSE	0.2877 m	0.3832 m	0.0256 m	0.00551 rad

These results match what we observed during testing. The vertical estimate is good because sonar and barometer help in different ways, and the yaw estimate is also accurate because the magnetometer gives a direct heading correction. The main remaining weakness is horizontal lag, but the estimator is still stable enough for the full mission.

4 Contribution

- **A0319170J:** Implemented the behavior node, including the seven-state FSM design, live waypoint tracking for the `TURTLE.POSITION` state, cycle-completion guarantee, and path availability guard. Tuned all behavior node parameters. Contributed to the report (Section 1: Behavior).
- **A0318793N:** Implemented the path-following controller, including lookahead point selection, proportional velocity control, velocity saturation, world-to-body frame transformation, and constant yaw rate command. Tuned all controller parameters. Contributed to the report (Section 2: Controller).
- **A0326802J:** Implemented the Kalman filter estimator, including state decomposition, IMU prediction, multi-sensor correction models, sonar gating, barometer bias state, GPS velocity correction, and GPS forward compensation. Tuned all estimator parameters. Contributed to the report (Section 3: Estimator).



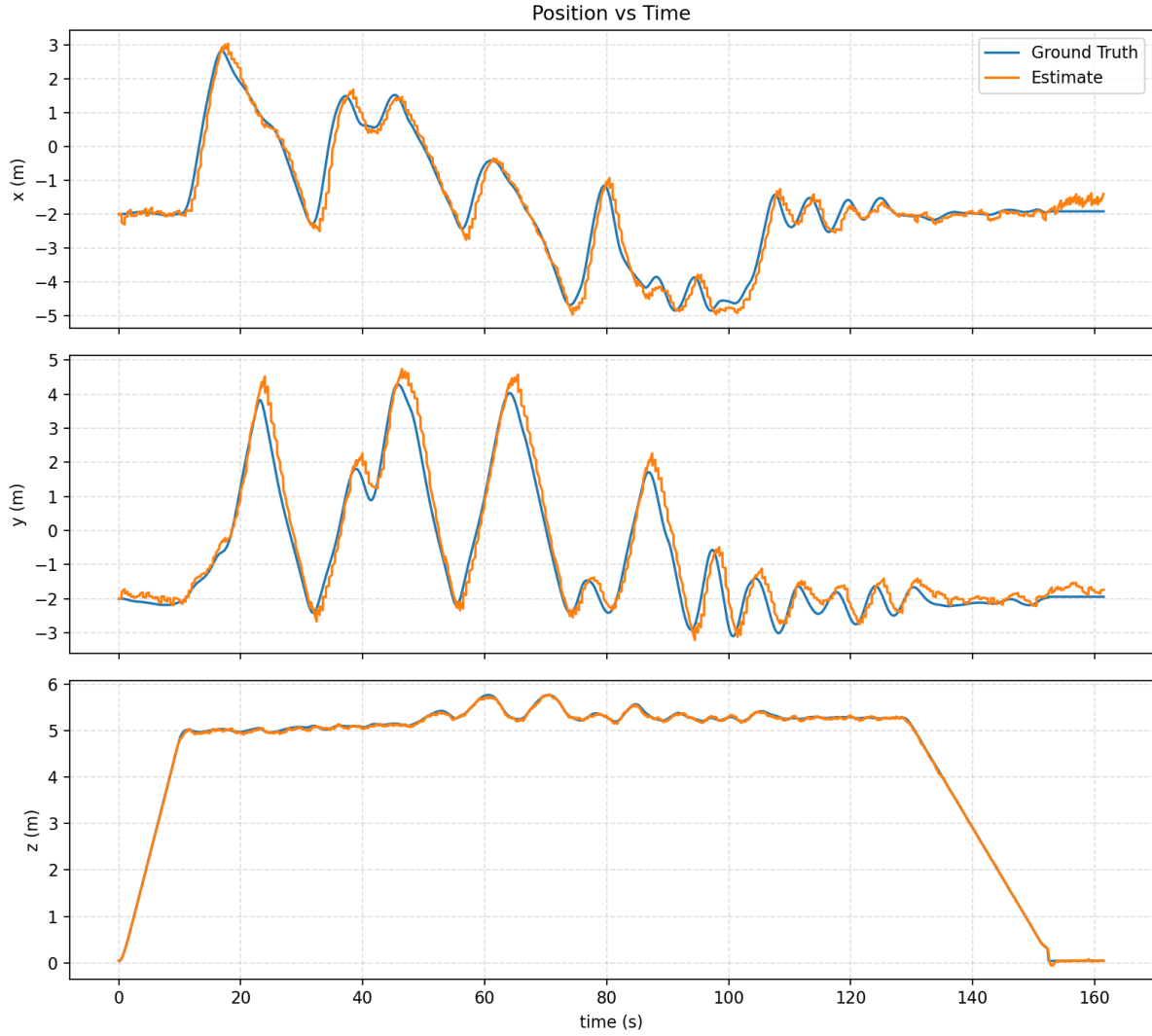


Figure 3: Position traces in the final validation experiment. The z estimate stays close to ground truth during takeoff, cruise, and landing, while the larger visible mismatch remains in the horizontal axes.

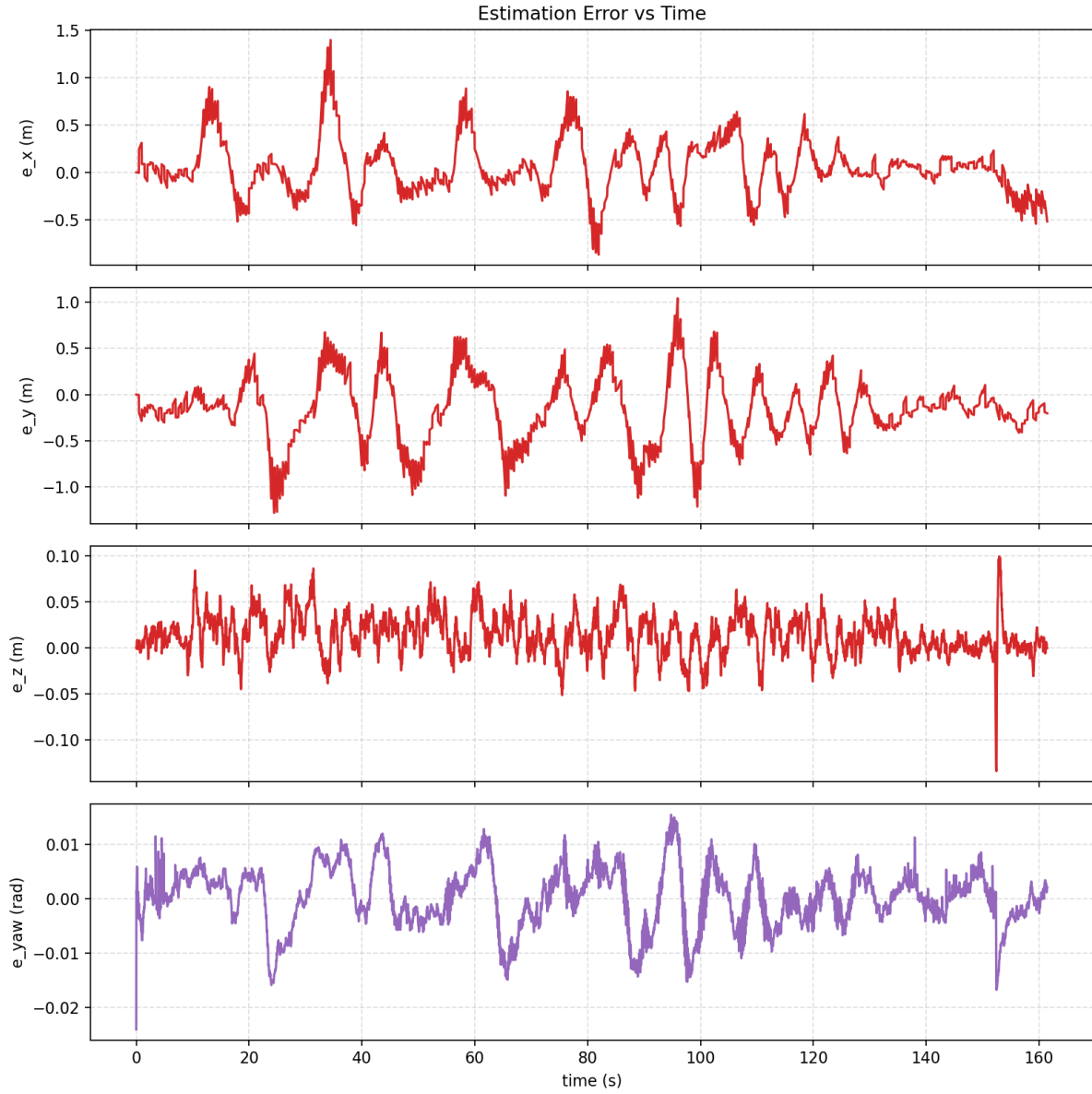


Figure 4: Estimation error in the final validation experiment. The z and yaw errors remain small throughout the mission, while the larger residual oscillations are concentrated in x and y .

- All three members jointly integrated and debugged the full drone mission stack in simulation, participated in code review, and contributed to the writing and proofreading of the report.

5 AI Tool Declaration

We used Sonnet 4.6 during the completion of this project.